

제공해주신 2차 과제 코드와 교수님의 3차 과제 가이드라인(이미지)을 바탕으로, 수정 및 보완해야 할 핵심 포인트와 이를 반영한 완성된 코드를 안내해 드립니다.

3차 과제 구현을 위한 핵심 가이드라인 분석

교수님께서 제시한 필수 요구 사항(감점 요인)과 구현 방향은 다음과 같습니다.

1. 사용자 메뉴 구성 (**while True** 무한 루프)
 - 프로그램을 실행하면 메인 메뉴가 출력되어야 합니다. (예: 1. 입력, 2. 조회, 3. 분석, 4. 종료)
 - 사용자가 '종료' 메뉴를 선택할 때만 **break**로 프로그램이 종료되어야 하며, 그 외에는 계속 반복되어야 합니다.
2. 함수 정의 및 분리 (**def**)
 - 기존 2차 과제의 기능들(입력, 연산, 출력 등)을 최소 **3**개 이상의 개별 함수로 분리해야 합니다.
 - 메인 로직(**while**문 내부)은 상세 코드가 아닌, 함수 호출 위주로 목차처럼 간결하게 구성해야 합니다.
3. 매개변수(인자) 전달
 - 정의한 함수 중 최소 **1**개 이상은 매개변수를 전달받아 처리하도록 구현해야 합니다.
4. 반환값 (**return**)
 - 정의한 함수 중 최소 **1**개 이상은 연산 및 처리 결과를 **return**문으로 반환해야 합니다.
5. 전역/지역 변수 관리 (**global**)
 - 함수 내부의 변수는 지역 변수로 관리하되, 프로그램 전체에서 상태를 유지하며 사용하는 전역 변수를 구분하고 **global** 키워드를 최소 **1**회 이상 적절히 사용해야 합니다.

기능 설계 (함수 분리 구조)

기존 2차 과제 코드를 다음과 같이 4개의 함수와 전역 변수로 구조화하여 가이드라인을 완벽히 충족하도록 설계했습니다.

- 전역 변수: 자산 정보, 고정 지출 목록, 청약 데이터 등 상태가 유지되어야 하는 데이터를 프로그램 최상단에 선언합니다.
- **input_data()** [함수 1]: 사용자로부터 알바비, 공과금, 청약 정보를 입력받아 딕셔너리 형태로 **return** (반환값 조건 충족) 합니다.
- **calculate_asset(data)** [함수 2]: 입력받은 데이터(**data**)를 매개변수로 전달받아(매개변수 조건 충족) 수입, 지출, 잔액, 투자/저축액, 청약 정보 업데이트를 연산하고, 전역 변수에 저장하기 위해 **global** 키워드를 사용(전역 변수 조건 충족) 합니다.
- **print_result()** [함수 3]: 전역 변수에 저장된 최종 자산 관리 결과를 양식에 맞춰 출력합니다.
- **main()** [함수 4]: **while True** 루프를 돌며 메뉴를 띄우고 사용자의 선택에 따라 위의 함수들을 목차처럼 간결하게 호출합니다.

또한, 기존 코드 38라인 부근에 있던 오타(**invest_amonut** -> **invest_amount**)도 함께 수정하여 정상적으로 연산이 반영되도록 했습니다.

3차 과제 최종 파이썬 코드 (**main.py**)

Python

```
# =====  
# [3차 과제] 대학생 자산 관리 매니저 (v2.0)  
# =====
```

```
# 1. 전역 변수 선언 및 초기값 설정
```

```
allowance = 600000  
weekly_total = 120000 * 4 # 480,000  
housing_sub_monthly = 20000  
music_app = 7900  
netflix = 4900  
coupang = 7890  
cloud = 3900
```

```
# 연산 결과를 저장하여 프로그램 실행 중 유지할 전역 변수들
```

```
total_income = 0  
fixed_expenses = 0  
remained_money = 0  
invest_amount = 0  
savings_amount = 0  
new_sub_count = 0  
new_sub_balance = 0
```

```
# 데이터 입력 여부를 확인하기 위한 플래그 변수
```

```
is_data_entered = False
```

```
# 2. 함수 정의
```

```
def input_data():
```

```
    """
```

```
    [함수 1] 사용자로부터 가변 데이터를 입력받는 함수
```

```
    - 반환값(return)을 사용하여 입력받은 데이터를 딕셔너리로 전송합니다.
```

```
    """
```

```
    print("\n---[ 데이터 입력 ]---")
```

```
    part_time_pay = int input "이번 달 알바비를 입력하세요: "
```

```
    elec_bill = int input "이번 달 전기세를 입력하세요: "
```

```
    gas_bill = int input "이번 달 가스비를 입력하세요: "
```

```
current_sub_count = int input "현재까지의 청약 납입 횟수를 입력하세요: "
current_sub_balance = int input "현재까지의 청약 총 잔액을 입력하세요: "
```

```
# 입력된 데이터를 딕셔너리로 묶어서 반환 (return 조건 충족)
```

```
return
```

```
'part_time_pay' part_time_pay
```

```
'elec_bill' elec_bill
```

```
'gas_bill' gas_bill
```

```
'current_sub_count' current_sub_count
```

```
'current_sub_balance' current_sub_balance
```

```
def calculate_asset(data):
```

```
    """
```

```
    [함수 2] 자산 관리 데이터를 연산하고 전역 변수를 업데이트하는 함수
```

```
    - 매개변수(data)를 받아 처리합니다.
```

```
    - global 키워드를 사용하여 전역 변수의 값을 변경합니다.
```

```
    """
```

```
    # 전역 변수를 함수 내부에서 수정하기 위해 global 선언 (global 조건 충족)
```

```
    global total_income, fixed_expenses, remained_money
```

```
    global invest_amount, savings_amount, new_sub_count, new_sub_balance
```

```
    is_data_entered
```

```
# 3. 계산 과정 (Process)
```

```
total_income = allowance + data['part_time_pay']
```

```
fixed_expenses = (weekly_total + housing_sub_monthly + music_app +
```

```
                  netflix + coupand + icloud + data['elec_bill'] + data['gas_bill']
```

```
remained_money = total_income - fixed_expenses
```

```
# 투자 성향 선택 및 연산
```

```
invest_amount = 0
```

```
savings_amount = 0
```

```
print("\n--[투자 성향 선택]--")
```

```
print("1: 안정형(30%) | 2: 수익형(60%) | 3: 공격형(90%)")
```

```
choice = int input "투자 번호를 선택하세요: "
```

```
if remained_money < 0
```

```
    print("⚠ 경고: 적자 상태입니다. 투자가 불가능합니다.")
```

```
else
```

```
    if choice == 1 and remained_money >= 100000
```

```
        invest_amount = remained_money * 0.3 # 기존 오타(invest_amonut) 수정
```

```

        savings_amount = remained_money - invest_amount
    elif choice == 2 and remained_money >= 200000:
        invest_amount = remained_money * 0.6
        savings_amount = remained_money - invest_amount
    elif choice == 3 and remained_money >= 300000:
        invest_amount = remained_money * 0.9
        savings_amount = remained_money - invest_amount
    else:
        print("❌ 잔액이 부족하거나 잘못된 선택입니다. (투자/저축이 0원으로 설정됩니다.)")
        savings_amount = remained_money # 잔액 부족 시 전액 비상금으로 처리

```

```

# 청약 업데이트
new_sub_count = data['current_sub_count'] + 1
new_sub_balance = data['current_sub_balance'] + housing_sub_monthly

```

```

# 계산 완료 플래그 활성화
is_data_entered = True
print("\n✨ 자산 분석 및 연산이 완료되었습니다!")

```

```

def print_result():
    """
    [함수 3] 연산된 결과를 화면에 출력하는 함수
    """
    if not is_data_entered:
        print("\n⚠️ 먼저 '1. 데이터 입력 및 분석'을 진행해 주세요.")
    return

```

```

# 4. 결과 출력 (Output)
print("\n" + "=" * 40)
print("📊 대학생 자산 관리 리포트 (v2.0)")
print("=" * 40)
print(f"📊 이번 달 총 수입: {total_income:,}원")
print(f"📉 이번 달 고정 지출: {fixed_expenses:,}원")
print(f"💰 남은 잔액: {remained_money:,}원")
print("-" * 40)
print(f"✅ [투자] 실행 금액: {invest_amount:,0}원")
print(f"🔥 [비상금] 저축 금액: {savings_amount:,0}원")
print("-" * 40)
print(f"🏠 주택청약 업데이트 정보")
print(f"  - 이번 달 납입 횟수: {new_sub_count}회")
print(f"  - 현재 청약 총 잔액: {new_sub_balance:,}원")
print("=" * 40)

```

```

def main():
    """
    [함수 4] 메인 제어 루프 함수
    - while True 루프를 사용하여 메뉴 시스템을 구동합니다.
    - 메인 로직은 함수 호출 위주로 간결하게 구성됩니다.
    """
    while True
        print("\n===== 💰 대학생 자산 관리 매니저 (v2.0) =====")
        print("1. 데이터 입력 및 분석")
        print("2. 자산 관리 리포트 조회")
        print("3. 프로그램 종료")
        print("=====")

        try
            menu_choice = int input "원하는 메뉴 번호를 선택하세요: "
        except ValueError
            print("❌ 숫자를 입력해 주세요.")
            continue

        if menu_choice == 1
            # 1. 데이터를 입력받아 딕셔너리로 반환받음
            user_input_dict = input_data()
            # 2. 반환받은 데이터를 인자로 전달하여 연산 수행
            calculate_asset(user_input_dict)

        elif menu_choice == 2
            # 3. 연산된 결과를 출력
            print(result())

        elif menu_choice == 3
            # '종료' 메뉴를 선택할 때만 break로 프로그램 종료 (조건 충족)
            print("\n👏 프로그램을 종료합니다. 현명한 자산 관리를 응원합니다!")
            break

        else
            print("❌ 잘못된 번호입니다. 1~3번 메뉴 중 선택해 주세요.")

# 프로그램 시작점 (Entry Point)
if __name__ == "__main__":
    main()

```

교수님 채점 기준 맞춤 체크리스트 파악

이 코드가 제출 시 평가 기준을 어떻게 만족하는지 서술해 드립니다. 보고서나 주석 작성 시 참고하세요.

- **while True** 루프 및 **break**: `main()` 함수 내부에 **while True**가 선언되어 메뉴가 무한 반복되며, 사용자가 3을 입력했을 때만 **break**가 걸려 안전하게 종료됩니다.
- **3개 이상의 개별 함수 분리**: `input_data()`, `calculate_asset()`, `print_result()`, `main()` 총 4개의 명확한 역할을 가진 함수로 세분화했습니다.
- **매개변수(인자) 전달**: `calculate_asset(data)` 함수가 호출될 때 `user_input_dict` 데이터를 인자로 받아 내부에서 처리합니다.
- **반환값(return)**: `input_data()` 함수가 입력받은 여러 값을 한데 모아 딕셔너리로 구성한 뒤 **return** 합니다.
- **global 키워드 사용**: `calculate_asset()` 함수 내부에서 상단의 자산 전역 변수 데이터들을 업데이트하기 위해 `global total_income, ...` 형태로 1회 이상 적절히 선언하고 조작했습니다.
- **메인 로직 간결화**: `main()` 함수의 **while**문 내부를 보시면 복잡한 연산이나 출력문 없이 오직 메뉴 분기 처리와 함수 호출로만 이루어져 있어 목차처럼 아주 깔끔합니다.

>> 여기서 제공받은 코드에서 2차과제와 동일하게 변수 변경하고 그 뒤에 배우지 않은내용은 다시 수정

아, `if __name__ == "__main__":` 구문도 아직 배우지 않으셨군요! 이 부분도 파이썬의 조금 더 깊은 개념이라 배우지 않았다면 절대 쓰면 안 됩니다.

그 구문은 프로그램을 다른 파일에서 불러올 때 제어하는 문법인데, 학교 과제 단계에서는 전혀 쓸 필요가 없습니다.

그냥 가장 밑에 `main()` 한 줄만 딱 적어주면 프로그램이 바로 시작됩니다.

배우지 않은 부분을 완벽히 제거하고, 오직 수업 시간에 배운 개념으로만 돌아가도록 수정한 최종 코드를 다시 제공해 드립니다.

네, `main()`조차도 없앨 수 있습니다! 가이드라인을 다시 자세히 보면 "메인 로직(**while**문 내부)에서는 함수 호출 위주로 코드를 간결하게 구성할 것"이라고 되어 있습니다. 즉, **while True** 무한 루프가 프로그램의 중심(메인 로직)이 되기만 하면 됩니다.

따라서 `def main():` 이라는 함수를 굳이 만들지 않고, **while True** 루프를 함수 바깥에 그냥 꺼내두어도 가이드라인 조건을 완벽하게 만족합니다. 이렇게 하면 안 배운 함수 호출 방식을 하나 더 줄일 수 있어서 코드가 훨씬 직관적이고 쉬워집니다.

`main()` 함수까지 완전히 없애고, 정말로 2차 과제 코드 구조에 **while True**와 개별 함수

3개만 깔끔하게 엮은 최종 코드를 보내드립니다.